The Model That Lost Alone and Won the Ensemble

An LLM as an orthogonal ranker for survival prediction

A case study: the ANNITIA 2026 challenge. Predict major hepatic events and all-cause death from irregular longitudinal records of MASLD patients (metabolic dysfunction-associated steatotic liver disease).

Every cell that runs here uses a synthetic cohort or our own aggregate scores. No challenge record appears in this notebook, and a guard enforces that.

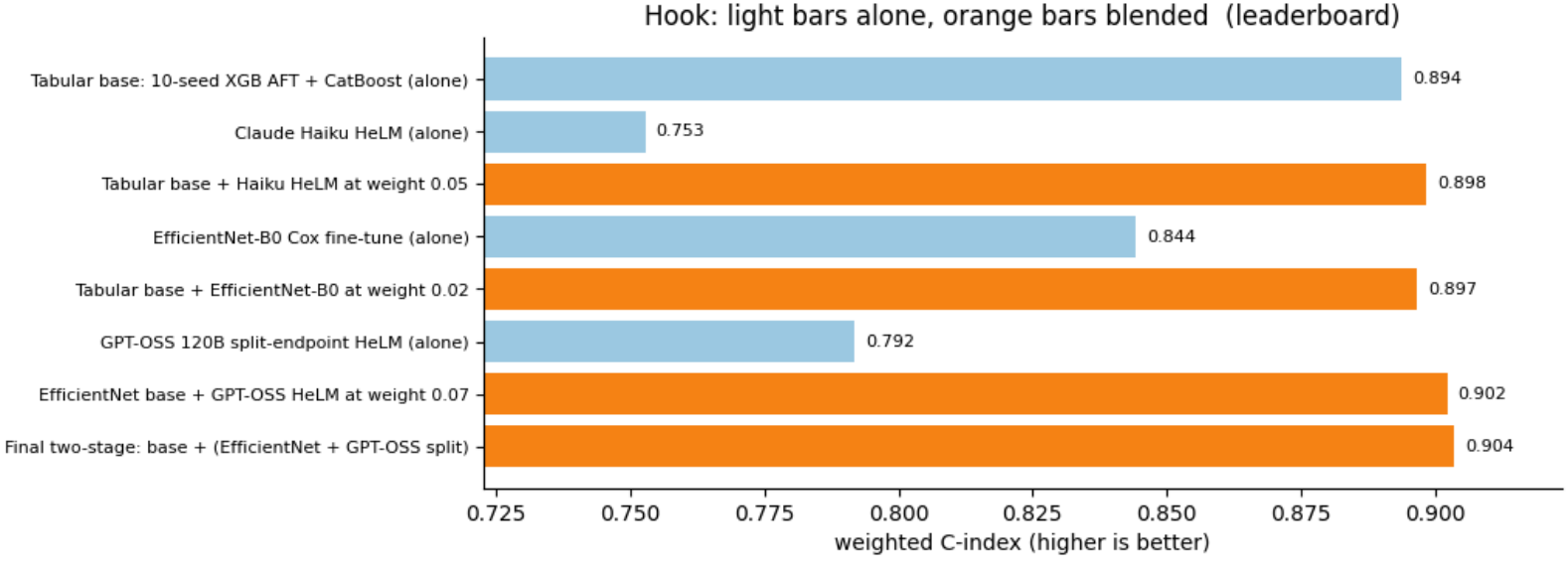
The argument, in one sentence

A reasoning LLM that reads a patient's history as clinical text is a **weak** survival model on its own. It still improves a strong tabular ensemble, because it makes **different mistakes**.

1. **Objective:** the metric only sees order, so we blend ranks.
2. **Data:** wide, irregular, censored records.
3. **Tabular base:** XGBoost AFT and CatBoost SurvivalAft.
4. **Serializing a patient:** from a wide row to a clinical summary.
5. **Why a weak ranker helps:** error correlation is the lever.
6. **Checking orthogonality** before spending API calls.
7. **The two-stage blend** we submitted.
8. **Limits** and honest reporting.

In [2]:

```
cv, lb = csv_rows("cv_scores.csv"), csv_rows("leaderboard.csv")
if lb:                                              # our own leaderboard milestones
    labels, vals, src = [r["submission"] for r in lb], [float(r["public_score"]) for r in lb], "leaderboard"
elif cv:                                            # our measured cross-validation numbers
    labels, vals, src = [r["model"] for r in cv], [float(r["weighted"]) for r in cv], "10-fold CV"
else:                                               # fallback: the same story on the synthetic cohort
    tab, llm = S.rankers(C.true_hep.to_numpy(), seed=1, shared=0.1)
    ev, t = C.hep_event, C.hep_time
    labels = ["tabular (alone)", "LLM (alone)", "tabular + LLM ranks"]
    vals = [cidx(ev, t, tab), cidx(ev, t, llm), cidx(ev, t, 0.8 * _rank01(tab) + 0.2 * _rank01(llm))]
    src = "synthetic cohort"
fig, ax = plt.subplots(figsize=(9, 3.8))
colours = ["#9ecae1" if "(alone)" in l else "#f58518" for l in labels]
ax.barh(labels[::-1], vals[::-1], color=colours[::-1])
for i, v in enumerate(vals[::-1]):
    ax.text(v + 0.002, i, f"{v:.3f}", va="center", fontsize=8)
ax.set_xlim(min(vals) - 0.03, max(vals) + 0.02); ax.set_xlabel("weighted C-index (higher is better)")
ax.tick_params(axis="y", labelsize=8)
ax.set_title(f"Hook: light bars alone, orange bars blended  ({src})")
plt.show()
```



On the leaderboard, GPT-OSS alone scored **0.792**, far below the tabular base at **0.894**. Blended at a small rank weight, it gave this pipeline's best score, **0.904**.

1 · Objective: the metric only sees order

Score = 0.7 × C-index(hepatic) + 0.3 × C-index(death).

Harrell's C-index asks one question of every comparable pair: did the patient with the earlier event get the higher risk? Scale, calibration and units are invisible to it.

In [3]:

```
tab, llm = S.rankers(C.true_hep.to_numpy(), seed=1, shared=0.1)
ev, t = C.hep_event, C.hep_time
print(f"C-index of a score            {cidx(ev, t, tab):.4f}")
print(f"C-index of exp(score)         {cidx(ev, t, np.exp(tab)):.4f}   (any monotone map: identical)")

# Averaging raw scores lets the wider scale win. Averaging ranks weighs both fairly.
raw = 0.5 * tab + 0.5 * llm                          # llm lives on 0-100, tab on about -3..3
print(f"\nraw 50/50 average    {cidx(ev, t, raw):.4f}   <- mostly the LLM, by scale alone")
print(f"rank 50/50 average   {cidx(ev, t, rank_blend({'tab': tab, 'llm': llm}, {'tab': .5, 'llm': .5})):.4f}")
```

```
C-index of a score            0.7926
C-index of exp(score)         0.7926   (any monotone map:
identical)

raw 50/50 average    0.7606   <- mostly the LLM, by scale
alone
rank 50/50 average   0.7884
```

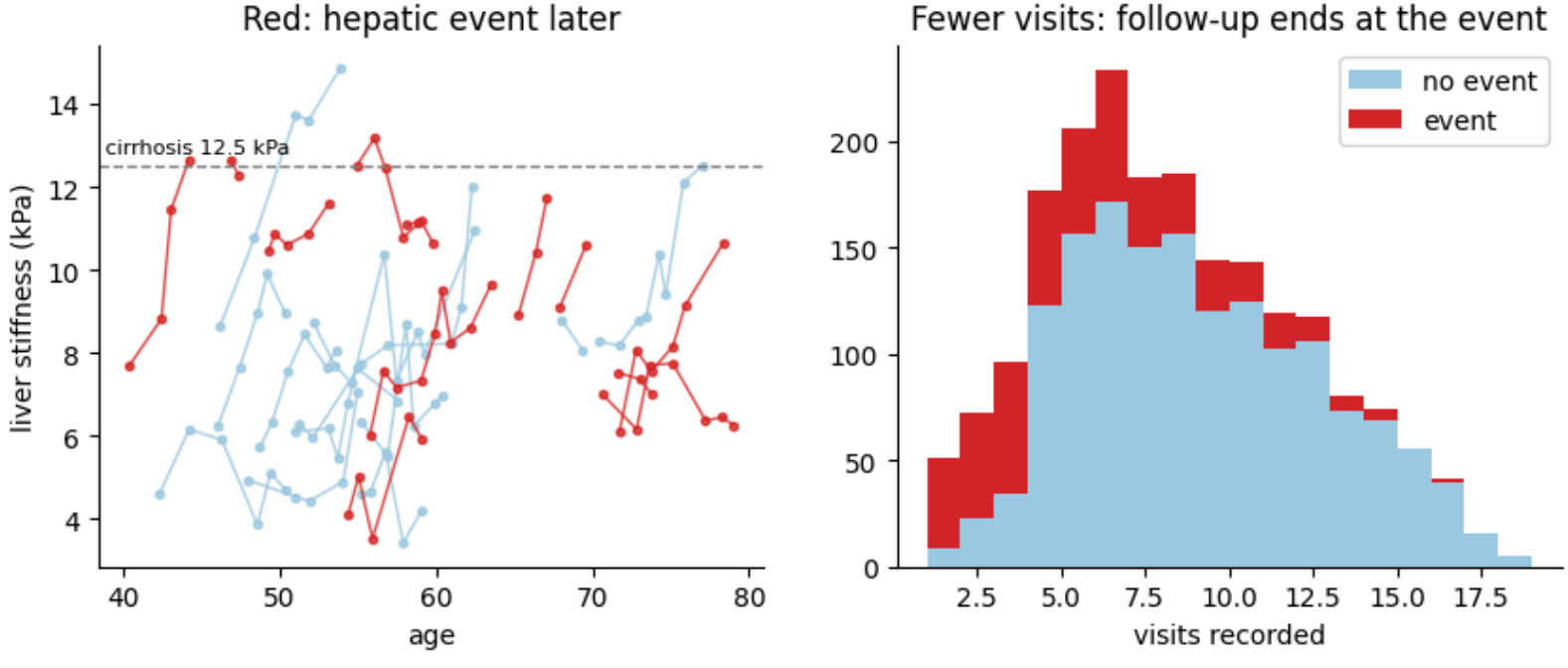
Transfer: when the metric is a ranking metric, blend ranks. Then models with incompatible outputs (log-times, hazards, 0-100 LLM scores) can be combined safely.

2 · The data: wide, irregular and censored

One row per patient, up to 22 visits, each with biomarkers such as ALT, platelets, FibroScan stiffness and FibroTest. Visits are irregular, non-invasive tests are often missing, and most patients never have the event (right censoring).

In [4]:

```
fig, (a1, a2) = plt.subplots(1, 2, figsize=(10, 3.6))
rng = np.random.default_rng(3)
for ev_flag, colour in ((False, "#9ecae1"), (True, "#d62728")):
    for i in rng.choice(np.flatnonzero(C.hep_event == ev_flag), 12, replace=False):
        r = C.iloc[i]
        ages = [r.get(f"Age_v{v}") for v in range(1, 23)]
        kpa = [r.get(f"fibs_stiffness_med_BM_1_v{v}") for v in range(1, 23)]
        pts = [(a, k) for a, k in zip(ages, kpa) if pd.notna(a) and pd.notna(k)]
        if pts:
            a1.plot(*zip(*pts), marker=".", color=colour, lw=1, alpha=0.8)
a1.axhline(12.5, ls="--", c="grey", lw=1); a1.text(a1.get_xlim()[0], 12.8, " cirrhosis 12.5 kPa", fontsize=8)
a1.set_xlabel("age"); a1.set_ylabel("liver stiffness (kPa)"); a1.set_title("Red: hepatic event later")
visits = C.filter(regex=r"^Age_v\d+$").notna().sum(axis=1)
a2.hist([visits[~C.hep_event], visits[C.hep_event]], bins=range(1, 20), label=["no event", "event"],
        color=["#9ecae1", "#d62728"], stacked=True)
a2.set_xlabel("visits recorded"); a2.legend(); a2.set_title("Fewer visits: follow-up ends at the event")
plt.show()
print(f"hepatic events {C.hep_event.mean():.1%} | deaths {C.death_event.mean():.1%} | censored for both "
      f"{(~C.hep_event & ~C.death_event).mean():.1%}")
```




```
hepatic events 23.1% | deaths 14.7% | censored for both
62.2%
```

Leakage trap: visit count and follow-up length encode the outcome. The pipeline drops these label proxies and masks visits after the event before building features.

3 · The tabular base: accelerated failure time boosting

XGBoost AFT and CatBoost SurvivalAft, rank blended per endpoint and averaged over 10 seeds. The real features are trajectory summaries (first, last, slope, volatility), clinical scores such as FIB-4 and APRI, and missingness patterns. Here is a trimmed version on the synthetic cohort.

In [5]:

```
PREFIXES = ("Age", "BMI", "alt", "ast", "ggt", "plt", "bilirubin", "gluc_fast", "triglyc",
            "fibs_stiffness_med_BM_1", "fibrotest_BM_2")
def features(df):
    rows = []
    for _, r in df.iterrows():
        f = {c: r[c] for c in ("gender", "T2DM", "Hypertension", "Dyslipidaemia")}
        for p in PREFIXES:                                  # the repo's own trajectory summary
            tr = _traj(r, p)
            f.update({f"{p}_{k}": tr[k] for k in ("first", "last", "slope")})
        f["fib4"] = _fib4(r)
        rows.append(f)
    return pd.DataFrame(rows, dtype=float)

X = features(C)
tr, te = np.arange(len(C)) < 1400, np.arange(len(C)) >= 1400
def fit_and_score(cols):
    Xtr, Xte, _, _ = transform_matrices(X.loc[tr, cols], X.loc[te, cols])   # the repo's imputer
    for ep in ("hep", "death"):
        ytr = ystruct(C[f"{ep}_event"][tr], C[f"{ep}_time"][tr])
        with contextlib.redirect_stderr(io.StringIO()):                     # quiet CatBoost bound notes
            p = {"xgb": fit_xgb(Xtr, ytr, seed=0)(Xte), "cat": fit_cat(Xtr, ytr, seed=0)(Xte)}
        e, t = C[f"{ep}_event"][te], C[f"{ep}_time"][te]
        print(f"  {ep:5s} XGB {cidx(e, t, p['xgb']):.3f}  Cat {cidx(e, t, p['cat']):.3f}"
              f"  blend {cidx(e, t, rank_blend(p, {'xgb': .5, 'cat': .5})):.3f}"
              f"  true risk {cidx(e, t, C[f'true_{ep}'][te]):.3f}")
whole_window = [c for c in X.columns if c.endswith(("_last", "_slope")) or c == "fib4"]
print("summaries over the whole follow-up window:"); fit_and_score(list(X.columns))
print("baseline visit only:");                       fit_and_score([c for c in X.columns if c not in whole_window])
```

```
summaries over the whole follow-up window:
  hep   XGB 0.914  Cat 0.921  blend 0.920  true risk 0.825
  death XGB 0.858  Cat 0.872  blend 0.871  true risk 0.722
baseline visit only:
  hep   XGB 0.793  Cat 0.798  blend 0.798  true risk 0.825
  death XGB 0.704  Cat 0.717  blend 0.717  true risk 0.722
```

A model that beats the **true risk** is reading the answer. Follow-up stops at the event, so last values, slopes and even whether a slope exists all encode how long follow-up lasted. The baseline-only model sits just under the truth, as an honest model should.

The real pipeline drops direct proxies (`follow_up_years` , `Age_delta`) and masks visits after the event. **Transfer:** compare every model against an oracle or a baseline-only

model.

4 · Serializing a patient for an LLM

The HeLM idea (Belyaeva et al., 2023): give a language model a patient's record as text and read a risk out of it. We turn each wide row into a short clinical summary: trajectories with direction words, reference thresholds and comorbidities.

In [6]:

```
n_visits = C.filter(regex=r"^Age_v\d+$").notna().sum(axis=1)
sick = C[C.hep_event & (n_visits >= 5)].sort_values("true_hep").iloc[-1]
import html
from IPython.display import HTML
lines = serialize_patient(sick, _fib4(sick)).splitlines()   # the exact function used for the submission
cut = lines.index("Liver biochemistry:")                    # two columns so it fits on one slide
col = lambda ls: f"<pre style='font-size:0.45em;margin:0'>{html.escape(chr(10).join(ls))}</pre>"
display(HTML(f"<div style='display:flex;gap:1.5em'>{col(lines[:cut])}{col(lines[cut:])}</div>"))
```

```
MASLD patient. Female, age 40.69–44.11 yr, 5 clinic visits.
Comorbidities: dyslipidaemia.
BMI: n=5, first=28.6 kg/m2, last=27.47 kg/m2, falling.

Non-invasive tests (NITs — liver fibrosis staging):
  FibroScan liver stiffness (kPa; F2=7.0, F3=9.5,
F4/cirrhosis=12.5): n=2, first=15.18 kPa, last=15.22 kPa, stable
  FibroTest composite score (0–1; F4 >0.74): n=2, first=0.91,
last=0.89, stable
  AIxplorer arterial stiffness: n=1, first=12.27, last=12.27
  FIB-4 index (low <1.30, high >2.67): 2.6
```

```
Liver biochemistry:
  ALT (U/L; ULN 40): n=5, first=74.39 U/L, last=82.12 U/L,
rising
  AST (U/L; ULN 40): n=5, first=56.95 U/L, last=69.17 U/L,
rising
  GGT (U/L): n=5, first=123.99 U/L, last=119.26 U/L, falling
  Bilirubin (umol/L): n=5, first=19.69, last=17.41, falling
  Platelets (G/L; thrombocytopenia <150): n=5, first=180.72 G/L,
last=129.49 G/L, falling

Metabolic markers:
  Fasting glucose (mmol/L): n=5, first=6.5, last=4.46, falling
  Triglycerides (mmol/L): n=5, first=2.27, last=1.51, falling
```

Split endpoints: one prompt per question

Hepatic events are driven by fibrosis. Death is driven mostly by age and cardiometabolic load. One combined prompt blurs the two, so each endpoint gets its own prompt and its own score.

In [7]:

```
print(SCORE_PROMPT_HEPATIC.split("---", 1)[1].format(cot_instruction="").strip())
```

```
You are a hepatologist assessing liver-specific risk for
this MASLD patient.
Provide a single continuous hepatic_risk score on a 0–100
scale.

This score reflects the probability of a major hepatic
event: ascites, hepatic encephalopathy, variceal bleed, HCC,
or liver transplant.

Key clinical signals for hepatic risk:
- FibroScan >12.5 kPa = cirrhosis (very high risk); >9.5 kPa
= F3 (high risk); >7.0 kPa = F2
- FIB-4 >2.67 = advanced fibrosis; FIB-4 1.30–2.67 =
intermediate
- FibroTest >0.74 = F4; >0.58 = F3
- Platelet decline below 150 G/L = portal hypertension sign
- Rising FibroScan slope = active fibrosis progression
- Persistently elevated ALT/GGT = ongoing hepatic injury
- T2DM accelerates fibrosis progression in MASLD
- Missing NITs (sparse follow-up) is a risk signal

Return only valid JSON — no other text:
{"hepatic_risk": <float 0–100>}
```

The model is GPT-OSS 120B, served through Ollama. Its 0-100 outputs are treated as **rankings**, not probabilities, which is all a C-index needs.

5 · Why a weak ranker helps: it is the error correlation

Two simulated rankers of the same true risk. The tabular one is strong. The LLM one is weaker and coarse (20 distinct values, like real LLM scores). We vary one knob: how correlated their mistakes are.

In [8]:

```
truth, ev, t = C.true_hep.to_numpy(), C.hep_event, C.hep_time
W = np.linspace(0, 1, 21)
curves = {}
for shared in (0.0, 0.3, 0.6, 0.9):
    tab, llm = S.rankers(truth, seed=7, shared=shared)
    c = [cidx(ev, t, (1 - w) * _rank01(tab) + w * _rank01(llm)) for w in W]
    curves[shared] = c; k = int(np.argmax(c))
    print(f"error corr {shared:.1f}: tabular {c[0]:.3f}  LLM {c[-1]:.3f}  best blend {c[k]:.3f}"
          f" at w={W[k]:.2f}  gain {c[k] - c[0]:+.4f}")
```

```
error corr 0.0: tabular 0.795  LLM 0.762  best blend 0.800
at w=0.30  gain +0.0049

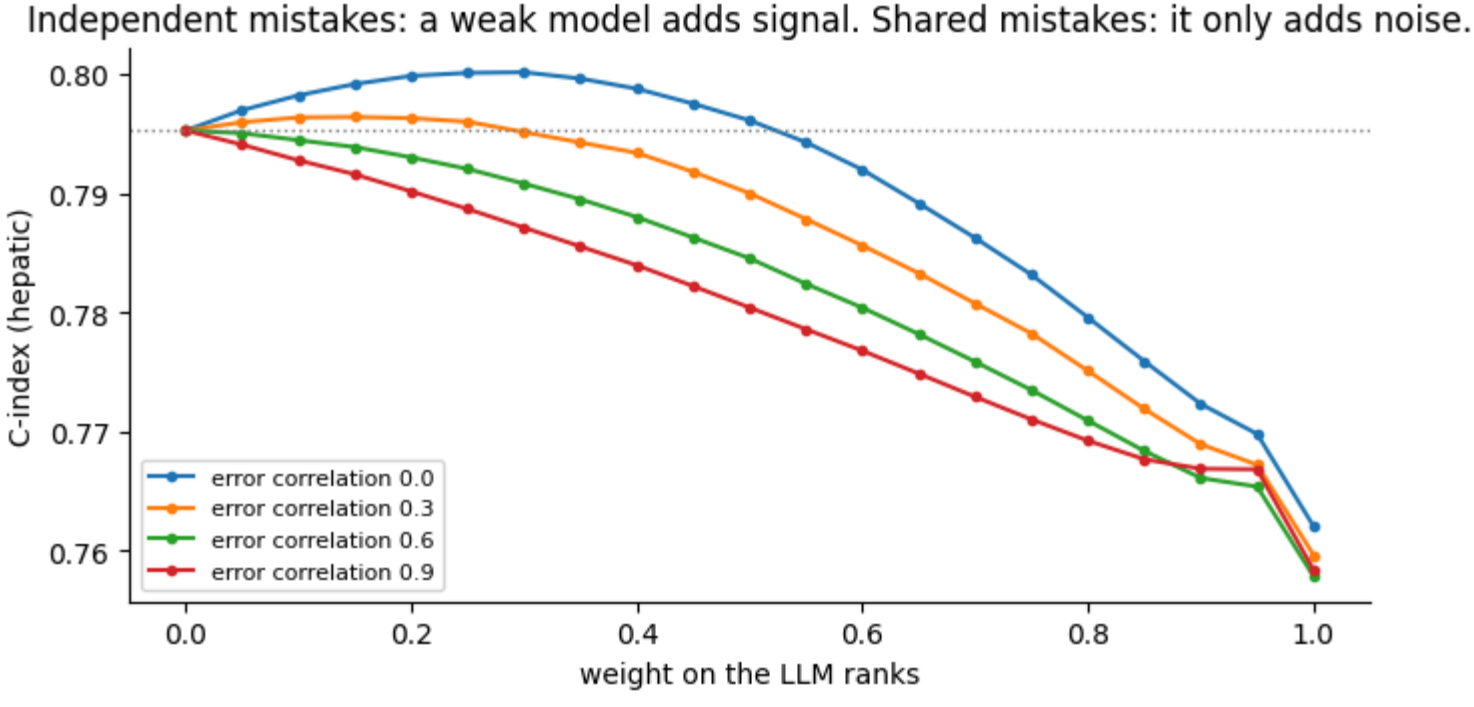
error corr 0.3: tabular 0.795  LLM 0.760  best blend 0.796
at w=0.15  gain +0.0011

error corr 0.6: tabular 0.795  LLM 0.758  best blend 0.795
at w=0.00  gain +0.0000

error corr 0.9: tabular 0.795  LLM 0.758  best blend 0.795
at w=0.00  gain +0.0000
```

In [9]:

```
for shared, c in curves.items():
    plt.plot(W, c, marker=".", label=f"error correlation {shared:.1f}")
plt.axhline(curves[0.0][0], ls=":", c="grey", lw=1)
plt.xlabel("weight on the LLM ranks"); plt.ylabel("C-index (hepatic)"); plt.legend(fontsize=8)
plt.title("Independent mistakes: a weak model adds signal. Shared mistakes: it only adds noise.")
plt.show()
```



Transfer: judge a candidate member by what it adds **to the ensemble**, not by its standalone score. A standalone gate would have thrown the LLM away.

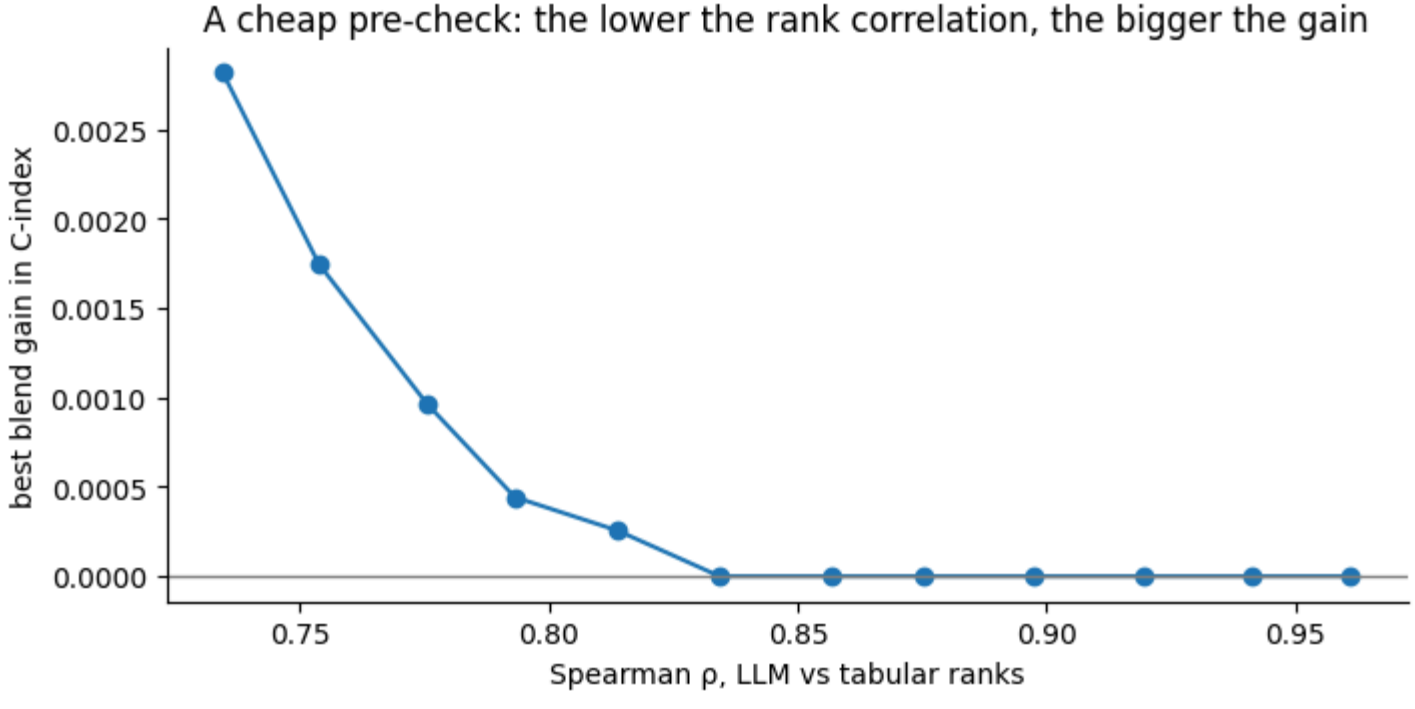
6 · Check orthogonality before spending API calls

Scoring every patient costs two LLM calls each. Two cheap diagnostics, both printed by our blend script, say early whether a new ranker is worth it:

- **Spearman ρ** between its ranks and the base ranks: lower means more new information.
- **Unique values:** a ranker with a handful of distinct scores ties most pairs.

In [10]:

```
rows = []
for shared in np.linspace(0, 0.95, 12):
    tab, llm = S.rankers(truth, seed=11, shared=shared)
    best = max(cidx(ev, t, (1 - w) * _rank01(tab) + w * _rank01(llm)) for w in W)
    rows.append((spearmanr(tab, llm)[0], best - cidx(ev, t, tab)))
rho, gain = zip(*rows)
plt.plot(rho, gain, marker="o"); plt.axhline(0, c="grey", lw=1)
plt.xlabel("Spearman ρ, LLM vs tabular ranks"); plt.ylabel("best blend gain in C-index")
plt.title("A cheap pre-check: the lower the rank correlation, the bigger the gain"); plt.show()
for levels in (5, 20, 100):
    _, llm = S.rankers(truth, seed=11, llm_levels=levels)
    print(f"{levels:3d} levels: {len(np.unique(llm)):3d} unique of {len(llm)}, LLM alone C = {cidx(ev, t, llm):.3f}")
```



```
  5 levels:   6 unique of 2000, LLM alone C = 0.743
 20 levels:  21 unique of 2000, LLM alone C = 0.751
100 levels: 101 unique of 2000, LLM alone C = 0.752
```

7 · The blend we submitted: two stages

1. **Stage 1:** an EfficientNet-B0 image and tabular model (Cox loss, SGD) blended 50/50 in rank space with the split-endpoint GPT-OSS scores.
2. **Stage 2:** the 10-seed XGBoost + CatBoost base, with stage 1 at weight 0.15.

Both stages are rank blends, so the LLM's effective weight is small:

In [11]:

```
w1, w2 = 0.50, 0.15                                # the weights in the README's reproduce steps
print(f"effective rank weights: base {1 - w2:.3f} | EfficientNet {w2 * (1 - w1):.3f} | GPT-OSS {w2 * w1:.3f}")

# The same recipe on synthetic rankers: base strong, image model medium, LLM weak and independent.
rng = np.random.default_rng(5)
base_r = truth + 0.6 * rng.normal(size=len(truth))
img_r = truth + 0.9 * (0.5 * (base_r - truth) / 0.6 + np.sqrt(0.75) * rng.normal(size=len(truth)))
_, llm_r = S.rankers(truth, seed=5, shared=0.1)
stage1 = (1 - w1) * _rank01(img_r) + w1 * _rank01(llm_r)
final = (1 - w2) * _rank01(base_r) + w2 * _rank01(stage1)
for name, s in (("base", base_r), ("image", img_r), ("LLM", llm_r), ("stage 1", stage1), ("final", final)):
    print(f"{name:8s} C = {cidx(ev, t, s):.4f}")
```

```
effective rank weights: base 0.850 | EfficientNet 0.075 |
GPT-OSS 0.075
base     C = 0.7886
image    C = 0.7733
LLM      C = 0.7699
stage 1  C = 0.7739
final    C = 0.7922
```

In [12]:

```
from IPython.display import Markdown
if cv or lb:
    md = "| source | model | score |\n|---|---|---|\n"
    md += "".join(f"| CV (measured) | {r['model']} | {float(r['weighted']):.4f} |\n" for r in cv)
    md += "".join(f"| leaderboard | {r['submission']} | {float(r['public_score']):.4f} |\n" for r in lb)
    display(Markdown("**Our real numbers**\n\n" + md))
else:
    display(Markdown("*Real numbers appear here after `python talk/export_scores.py` has run on the "
                     "challenge data. Leaderboard scores go in `talk/data/leaderboard.csv`.*"))
```

Our real numbers

source	model	score
leaderboard	Tabular base: 10-seed XGB AFT + CatBoost (alone)	0.8938
leaderboard	Claude Haiku HeLM (alone)	0.7528
leaderboard	Tabular base + Haiku HeLM at weight 0.05	0.8983
leaderboard	EfficientNet-B0 Cox fine-tune (alone)	0.8443
leaderboard	Tabular base + EfficientNet-B0 at weight 0.02	0.8966
leaderboard	GPT-OSS 120B split-endpoint HeLM (alone)	0.7919
leaderboard	EfficientNet base + GPT-OSS HeLM at weight 0.07	0.9022
leaderboard	Final two-stage: base + (EfficientNet + GPT-OSS split)	0.9036

8 · Limits and honest reporting

- **Uncalibrated:** the LLM scores rank patients. They are not risks, and are never reported as such.
- **Cost and drift:** two calls per patient, and a hosted model can change under you. Store the raw scores and the model tag with each run.
- **Unmeasured locally:** GPT-OSS was never run on the training set, so its blend gain has no CV estimate. Label it as a leaderboard number, not a validated one.
- **Weight selection bias:** a blend weight picked on a small split looks better than it is.

In [13]:

```
tab, llm = S.rankers(truth, seed=13, shared=0.3)
c_of = lambda idx, w: cidx(ev.iloc[idx], t.iloc[idx], (1 - w) * _rank01(tab[idx]) + w * _rank01(llm[idx]))
rng = np.random.default_rng(13); picked, val_gain, test_gain = [], [], []
for _ in range(40):                                  # tune the weight on 150 patients, test on the rest
    idx = rng.permutation(len(truth)); v, rest = idx[:150], idx[150:]
    k = int(np.argmax([c_of(v, w) for w in W])); picked.append(W[k])
    val_gain.append(c_of(v, W[k]) - c_of(v, 0)); test_gain.append(c_of(rest, W[k]) - c_of(rest, 0))
print(f"weights picked range {min(picked):.2f} to {max(picked):.2f}")
print(f"mean gain on the tuning split {np.mean(val_gain):+.4f} | on held-out patients {np.mean(test_gain):+.4f}")
```

```
weights picked range 0.00 to 0.50
mean gain on the tuning split +0.0034 | on held-out patients
-0.0014
```

Practice: fix a small grid, prefer round weights, and trust a gain only where held-out data or the leaderboard confirm it.

Generalising the idea

Setting	Strong base	Orthogonal ranker	Combine by
Clinical survival (this talk)	gradient-boosted AFT	LLM reading a clinical summary	rank blend
Credit or churn risk	boosted trees on tabular data	LLM over notes or transcripts	rank blend
Search and retrieval	a learned ranker	an LLM relevance judge	reciprocal rank fusion
Triage and alerting	rules and thresholds	an LLM reading free-text reports	ordered escalation

The checklist

- [ ] Is the metric a ranking metric? Then blend ranks, not raw scores.
- [ ] Did I drop label proxies and mask data recorded after the event?
- [ ] Does the serialized text carry trajectories and thresholds, not only last values?
- [ ] Is each endpoint asked separately?
- [ ] Is the candidate judged by its gain **in the ensemble**, not alone?
- [ ] Did I check rank correlation and unique values before paying for every call?
- [ ] Were blend weights fixed on a coarse grid and confirmed on held-out data?
- [ ] Is every number labelled measured, leaderboard or synthetic?